

Градиентный бустинг

Идея, математика и самостоятельная
реализация
для задачи регрессии

Основы машинного обучения

Вайндрах Елизавета,
Чернорицкая Анастасия
5030102/30202

Что такое градиентный бустинг

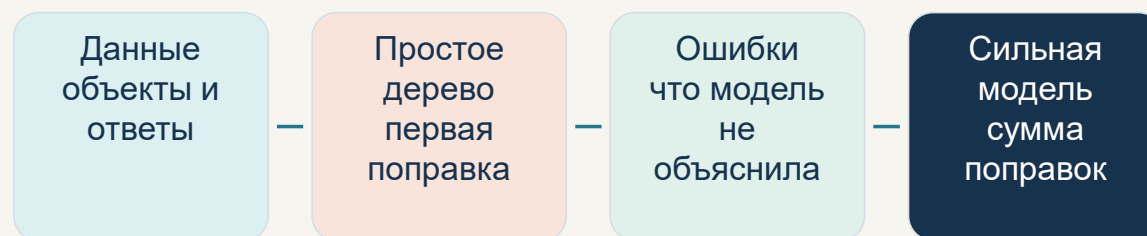
02

Ансамбль простых моделей, который последовательно уменьшает ошибку

Градиентный бустинг строит сильную модель постепенно: каждое новое небольшое дерево смотрит на ошибки текущего ансамбля и добавляет поправку.

Главная идея

Не пытаться сразу построить одно сложное дерево, а складывать много простых корректировок.



Базовые модели

небольшие решающие деревья

Задачи

регрессия и классификация

Ключевой принцип

каждое дерево исправляет предыдущие ошибки

Постановка задачи

03

По обучающей выборке восстановить зависимость между объектом и ответом

$$D = \{ (x_i, y_i) \}_{i=1}^N, \quad y_i \approx f(x_i)$$

x_i — признаки объекта
 y_i — правильный ответ
 $F(x)$ — прогноз модели

Для регрессии выбираем функцию потерь
и хотим сделать её средней по выборке
как можно меньше:

$$Q(F) = \frac{1}{N} \cdot \sum_{i=1}^N L(y_i, F(x_i)) \rightarrow \min$$

Табличные данные

Градиентный бустинг особенно полезен, когда признаки имеют разную природу и между ними есть нелинейные зависимости.

Цель обучения — уменьшить $Q(F)$

Аддитивная модель и жадное построение

04

Бустинг представляет сложную зависимость как сумму простых базовых алгоритмов

$$F^M(x) = F_0(x) + \sum_{m=1}^M \gamma_m \cdot h_m(x)$$

F_0 — начальное приближение
 h_m — новое решающее дерево
 γ_m — размер шага

На каждом шаге добавляется только один новый алгоритм.

Он выбирается так, чтобы улучшить уже построенную композицию:

$$(\gamma_m, h_m) = \arg \min \sum_{i=1}^N L(y_i, F_{m-1}(x_i) + \gamma \cdot h(x_i))$$

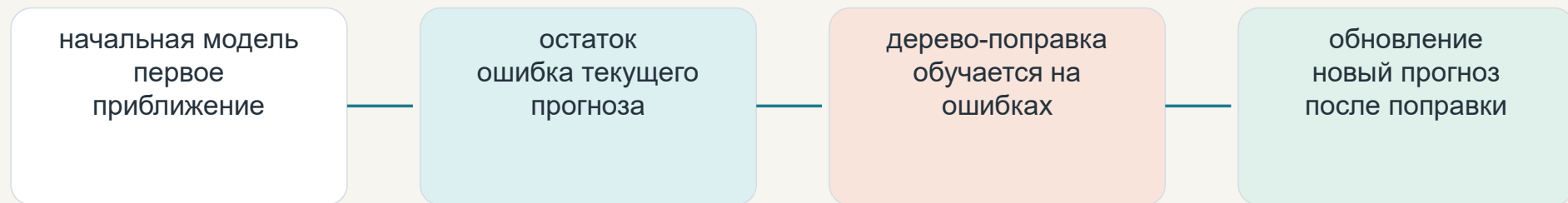
Последовательно, а не независимо

В этом отличие бустинга от бэггинга: следующий базовый алгоритм зависит от ошибок текущего ансамбля.

Интуиция: каждый шаг приближает прогноз к цели

05

Следующее дерево учится компенсировать ошибки текущей композиции



если $F_0(x_i) = y_i + 10$, то идеальная поправка равна -10

На практике дерево приближает эту поправку не идеально.
Поэтому добавляется следующий базовый алгоритм.

Квадратичная функция потерь

Сначала разберём частный случай, где антиградиент имеет простой вид

$$L(y, F(x)) = \frac{1}{2} \cdot (y - F(x))^2$$

$$Q(F) = \frac{1}{N} \cdot \sum_{i=1}^N L(y_i, F(x_i))$$

Производная по предсказанию модели:

$$\frac{\partial L}{\partial F} = F(x) - y$$

Направление уменьшения потерь — антиградиент:

$$-\frac{\partial L}{\partial F} = y - F(x)$$

Для MSE антиградиент = остаток

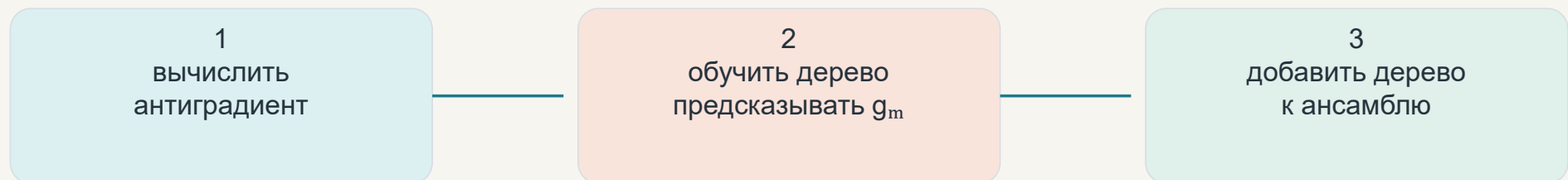
Обобщение: обучаемся на антиградиенте

07

Тот же принцип работает для любой дифференцируемой функции потерь

$$g_{i,m} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F} \right]_{(F = F_{m-1})}$$

На шаге m для каждого объекта вычисляем число $g_{i,m}$ — компоненту антиградиента по текущему предсказанию.



$$F_m(x) = F_{m-1}(x) + \gamma_m \cdot h_m(x)$$

Почему возникает антиградиент

Первое приближение функции потерь приводит к направлению шага

$$L(y_i, F_{m-1} + \gamma h) \approx L(y_i, F_{m-1}) + \gamma \cdot \left(\frac{\partial L}{\partial F} \right) \cdot h$$

Суммируем по объектам и убираем постоянную часть:

$$Q(F_{m-1} + \gamma h) \approx Q(F_{m-1}) + \gamma \cdot \sum_i \frac{\partial L_i}{\partial F_i} \cdot h(x_i)$$

$$g_{i, m} = - \frac{\partial L_i}{\partial F_i}$$

$$Q(F_{m-1} + \gamma h) \approx Q(F_{m-1}) - \gamma \cdot \langle g_m, h \rangle$$

Чтобы уменьшить Q , новое дерево h_m должно быть направлено примерно как вектор антиградиента g_m .

Как обучается очередное дерево

Для регрессии дерево приближает значения антиградиента

$$h_m \approx g_m, \quad \text{где } g_{i,m} = y_i - F_{m-1}(x_i) \text{ для } MSE$$

Для каждого кандидата на разбиение ищем средние значения в левом и правом листе:

$$c^L = \frac{1}{|L|} \cdot \sum_{i \in L} r_i, \quad c^R = \frac{1}{|R|} \cdot \sum_{i \in R} r_i$$

Качество разбиения измеряется суммой квадратов ошибок внутри листьев:

$$SSE = \sum_{i \in L} (r_i - c^L)^2 + \sum_{i \in R} (r_i - c^R)^2 \rightarrow \min$$

выбрать лучший порог —

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Численный пример: маленький набор данных

10

Используем те же данные, что и в самостоятельной реализации

Обучающая выборка после разбиения

x	y
1	1.0
2	1.4
4	2.7
5	3.0
7	4.2
9	5.4
10	6.3
11	6.7
12	7.5

$$F_0(x) = \bar{y} = \frac{(1.0 + \dots + 7.5)}{9} = 4.244$$

До обучения деревьев модель предсказывает одно и то же значение для всех объектов.

Начальный прогноз: 4.244

Три объекта оставлены для тестирования: x = 8, 6 и 3.

Численный пример: первый решающий пенъ

11

Первое дерево обучается на остатках начальной модели

$$r_{i,1} = y_i - F_0(x_i)$$

группа	объекты x	среднее y	средний остаток
левая	1, 2, 4, 5, 7	2.460	-1.784
правая	9, 10, 11, 12	6.475	+2.231
порог	$x \leq 8 / x > 8$		
ошибка	SSE внутри листьев		минимальна
дерево	$h_1(x)$		две поправки

$$h_1(x) = -1.784, \quad x \leq 8$$

$$h_1(x) = +2.231, \quad x > 8$$

Это значения поправки, а не итоговые прогнозы.

$$\eta \cdot h_1(x) = 0.1 \cdot h_1(x)$$

Численный пример: обновление прогноза

12

После первого дерева модель получает новые остатки

$$F_1(x) = F_0(x) + 0.1 \cdot h_1(x)$$

$$x \leq 8$$

$$F_1 = 4.244 + 0.1 \cdot (-1.784) = 4.066$$

$$x > 8$$

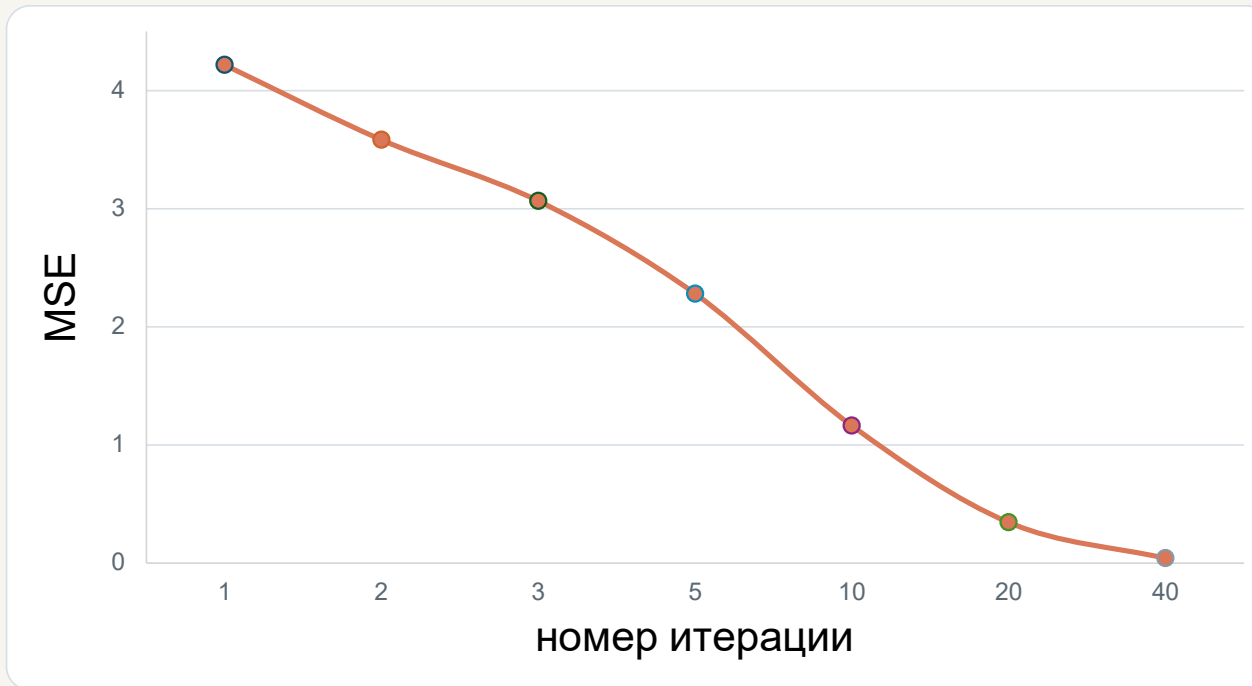
$$F_1 = 4.244 + 0.1 \cdot 2.231 = 4.467$$

Теперь второе дерево снова смотрит на остатки,
но уже после обновления F_1 .

$$h_2(x): x \leq 6 \rightarrow -2.041; \quad x > 6 \rightarrow +1.633$$

Численный пример: ошибка уменьшается

MSE на обучающей выборке после добавления новых деревьев



Интерпретация

- каждое дерево добавляет небольшую коррекцию
- обучающая ошибка падает с 4.2151 до 0.0404
- слишком много деревьев может привести к переобучению

40 деревьев · $\eta = 0.1$

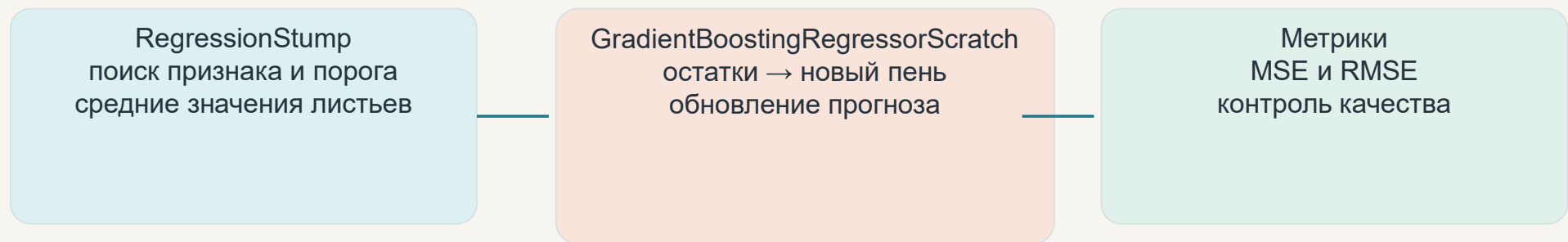
Данные графика: результат запуска собственной реализации

Самостоятельная реализация на Python

14

На слайде — структура решения, код демонстрируется отдельно

В основной реализации не используются готовые модели sklearn



- задача: регрессия
- базовые модели: решающие пни глубины 1
- функция потерь: MSE
- обучение: 40 итераций, скорость шага 0.1

Результаты запуска собственной модели

15

Код возвращает качество на обучающих и тестовых данных

Метрика	Обучение	Тест
MSE	0.0404	0.4866
RMSE	0.2011	0.6975

$$RMSE = \sqrt{MSE}$$

Тестовые предсказания

x	y	$\hat{y}$
8	5.1	4.213
6	3.8	3.029
3	1.8	1.519

На тесте ошибка выше, потому что модель не видела эти объекты во время обучения. Кроме того, тестовая выборка состоит всего из трёх точек.

Результат объясним и воспроизводим

Гиперпараметры и переобучение

Качество зависит не только от самой идеи бустинга

Параметр	Что контролирует	Слишком большое значение
n_estimators	число деревьев	переобучение, дольше обучение
learning_rate	вклад одного дерева	быстрые, резкие поправки
max_depth	сложность дерева	дерево запоминает шум
subsample	доля данных на шаге	больше разброса результата

Типичный компромисс

$\eta \downarrow \rightarrow$ обычно нужно $M \uparrow$

Маленькие шаги часто дают более устойчивое обучение, но требуют большего числа деревьев.

контроль: валидация и early stopping

Альтернативы и сравнение

17

Градиентный бустинг — один из нескольких ансамблевых подходов

Метод	Как строятся модели	Точность на таблицах	Интерпретируемость	Главный риск
Одно дерево	одна модель	средняя	высокая	недообучение или переобучение
Случайный лес	параллельно + усреднение	высокая	средняя	большой ансамбль
AdaBoost	усиливает ошибки объектов	высокая	средняя	чувствительность к выбросам
Gradient Boosting	последовательно + градиент	очень высокая	средняя	переобучение
XGBoost / LightGBM / CatBoost	оптимизированные варианты	очень высокая	ниже	сложнее настройка

Для учебного примера выбран классический градиентный бустинг: его шаги проще показать вручную.

Сильные стороны, ограничения и применение

18

Где метод особенно уместен, а где стоит быть осторожным

Сильные стороны

- высокое качество на табличных данных
- подходит для регрессии и классификации
- учитывает нелинейные зависимости
- не требует сложного преобразования признаков

Ограничения

- чувствителен к гиперпараметрам
- может переобучаться
- обучается последовательно и не всегда быстро
- отдельные деревья понятны, вся сумма сложнее для объяснения

Практические задачи

- оценка цены и спроса
- скоринг и риск
- прогнозирование оттока
- ранжирование объектов

Чаще всего: структурированные табличные данные

- 01 Бустинг складывает много слабых моделей последовательно.
- 02 Каждое новое дерево обучается на остатках предыдущей модели.
- 03 Для MSE отрицательный градиент совпадает с остатком.
- 04 Собственная реализация показывает механику без готовой модели sklearn.
- 05 Точность требует настройки числа деревьев, глубины и learning_rate.

$$F^M(x) = F_0(x) + \eta \cdot \sum_{m=1}^M h_m(x)$$

Литература и источники

20

Материалы, использованные для теории и практической части

Теория

- Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics, 2001. DOI: 10.1214/aos/1013203451
- Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning. 2nd ed. Springer, 2009.
- James G., Witten D., Hastie T., Tibshirani R. An Introduction to Statistical Learning. 2nd ed. Springer, 2021.

Онлайн-источники и код

- Яндекс Образование. Хэндбук по ML: «Градиентный бустинг» — <https://education.yandex.ru/handbook/ml/article/gradientnyj-busting>
- scikit-learn User Guide: Gradient-boosted trees — <https://scikit-learn.org/stable/modules/ensemble.html>
- scikit-learn example: Gradient Boosting regression — https://scikit-learn.org/stable/auto_examples/ensemble/plot_gradient_boosting_regression.html
- Собственная реализация : gradient_boosting_from_scratch.py

Спасибо за внимание